

A PROJECT
ON
**Design and Implementation of a Custom
Computer Processor Architecture Using
Logisim**

Submitted By

Name: Pranto Ghosh

ROLL : 2307075

Department: Computer Science and Engineering (CSE)

Course Title : Computer Architecture

Course No : CSE 2114

Semester: Second Year – First Semester

Submitted To:

MD. BADIUZZAMAN SHUVO

LECTURER,
COMPUTER SCIENCE AND ENGINEERING
KHULNA UNIVERSITY OF ENGINEERING AND
TECHNOLOGY

MR. MD. MUBTASHIM ABRAR NIHAL

LECTURER,
COMPUTER SCIENCE AND ENGINEERING
KHULNA UNIVERSITY OF ENGINEERING AND
TECHNOLOGY

Abstract

This project presents the design and implementation of a custom processor architecture using Logisim Evolution. The processor was developed as a practical implementation of fundamental concepts of Computer Organization and Architecture, including Arithmetic Logic Unit (ALU) design, instruction execution, control logic generation, register transfer operations, memory organization, and micro-operations.

The processor was designed from scratch and gradually extended by integrating multiple arithmetic and logical operations such as ADD, SUB, XOR, Increment, Decrement, and Shift operations. Later, a Booth Multiplication circuit was also integrated to extend the computational capability of the processor. The project uses ROM-based control logic to generate micro-operations and execute instructions sequentially through clock-driven operations.

Throughout the development process, several architectural and timing-related problems were encountered, especially during the integration of multiplication, accumulator loading, control sequencing, clock synchronization, instruction decoding, and shift implementation. These problems were analyzed and solved through iterative debugging and redesign.

The final processor demonstrates the complete flow of instruction execution starting from instruction fetch, instruction decode, control signal generation, ALU execution, data transfer, and result storage. This project provided practical knowledge about how a real processor operates internally and strengthened understanding of digital logic design, processor organization, and hardware-level computation.

Table of Contents

1. Introduction
2. Objectives
3. Background Theory
4. Project Overview
5. System Architecture
6. Processor Components
7. Instruction Set Design
8. ALU Design and Operations
9. ROM-Based Control Unit
10. Booth Multiplication Integration
11. Clock and Timing Mechanism
12. Problems Faced During Development
13. Solutions and Design Improvements
14. Working Procedure of the Processor
15. Advantages of the System
16. Limitations
17. Future Improvements
18. Conclusion
19. References

1. Introduction

Computer Architecture is one of the most important fields of Computer Science and Engineering because it explains how computers actually work internally. While software development focuses on applications and algorithms, computer architecture focuses on hardware organization, processor structure, instruction execution, memory systems, and control logic.

In modern processors, millions of operations are performed every second. Behind these operations lies a combination of arithmetic circuits, control units, registers, memory systems, buses, and synchronization mechanisms. Understanding these internal operations theoretically is difficult without practical implementation.

The main purpose of this project was to design a simplified but functional processor architecture using Logisim Evolution. Instead of only studying theoretical processor diagrams, this project aimed to implement a real working processor capable of executing instructions.

The processor was built incrementally. Initially, basic arithmetic operations such as ADD and SUB were implemented. After successful execution of these instructions, additional functionalities including XOR, Increment, Decrement, and Shift operations were added. Finally, a Booth Multiplication circuit was integrated into the architecture.

The project demonstrates how machine instructions are decoded and converted into low-level control signals through ROM-based control logic. It also demonstrates how data moves between registers, how ALU operations are performed, and how timing and synchronization affect processor behavior.

2. Objectives

The major objectives of this project are listed below:

- To understand the internal structure of a processor.
- To design a custom Arithmetic Logic Unit (ALU).
- To implement instruction execution using control logic.
- To generate micro-operations using ROM.

- To understand register transfer operations.
- To design instruction formats and opcode structures.
- To integrate arithmetic, logical, and shift operations.
- To implement Booth Multiplication in a processor architecture.
- To study clock synchronization and timing behavior.
- To gain practical experience with digital circuit design using Logisim Evolution.
- To understand the challenges of processor implementation and debugging.

Background Theory

Computer Architecture

Computer Architecture refers to the logical structure and functional behavior of a computer system. It includes instruction sets, memory organization, processor structure, control mechanisms, and data flow.

Computer Organization

Computer Organization focuses on the implementation of architectural features. It includes hardware-level details such as buses, control signals, registers, and ALU organization.

Arithmetic Logic Unit (ALU)

The ALU is one of the core components of a processor. It performs arithmetic and logical operations such as:

- Addition
- Subtraction
- XOR
- Increment
- Decrement
- Shift operations
- Multiplication

The ALU receives operands from registers and generates results according to the selected control signals.

Registers

Registers are small high-speed storage locations inside the processor. They temporarily hold data, instructions, addresses, and intermediate results.

Control Unit

The control unit controls the operation of the processor. It generates control signals required for instruction execution.

In this project, ROM-based control logic was used to generate micro-operations.

Micro-operations

Micro-operations are low-level operations performed on data stored in registers. Examples include:

- Load
- Store
- Increment
- Shift
- Transfer
- Arithmetic execution

Multiple micro-operations are combined to execute a complete instruction.

Booth Multiplication Algorithm

Booth's Algorithm is an efficient signed binary multiplication algorithm. It reduces the number of addition and subtraction operations required during multiplication.

The algorithm works by analyzing adjacent bits of the multiplier and performing:

- Addition
- Subtraction
- Arithmetic shift

This algorithm is commonly used in processor design because it improves multiplication efficiency.

Project Overview

The project implements a custom processor architecture using Logisim Evolution. The processor consists of:

- Arithmetic Logic Unit (ALU)
- Registers
- Control Unit
- ROM-based micro-operation generator
- Instruction Decoder
- Clock mechanism
- Multiplication module
- Data buses and control lines

The processor executes instructions sequentially through clock pulses.

The complete instruction execution process follows these stages:

1. Fetch instruction
2. Decode opcode
3. Generate control signals
4. Execute ALU operation
5. Store result

Initially, the processor supported only ADD and SUB instructions. Later, the following operations were integrated:

- XOR
- Increment
- Decrement
- Shift operations
- Booth Multiplication

This gradual integration made the processor more capable and closer to a real CPU structure.

System Architecture

The architecture of the processor follows a simplified CPU design model.

Major Components

1. Instruction Register (IR)

Stores the currently executing instruction.

2. Control Unit

Decodes instructions and generates control signals.

3. ROM

Stores micro-operations required for instruction execution.

4. ALU

Performs arithmetic and logical operations.

5. Registers

Store operands and intermediate values.

6. Accumulator

Stores ALU output results.

7. Clock System

Controls sequential execution.

8. Booth Multiplier

Performs signed binary multiplication.

Processor Components

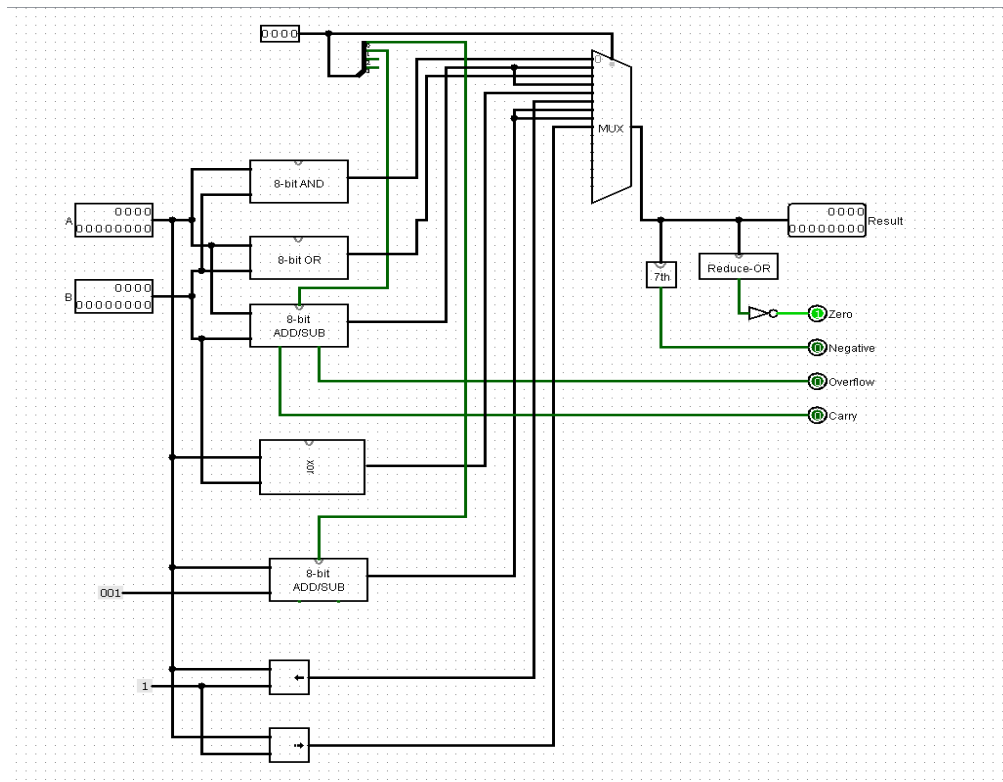
Arithmetic Logic Unit (ALU)

The ALU is responsible for executing arithmetic and logical operations

Operations Implemented

OPERATION	DESCRIPTION
ADD	Adds two operands
SUB	Subtracts operands
XOR	Performs bitwise XOR
INC	Increments value by 1
DEC	Decrements value by 1
SHIFT	Performs shift operation
MUL	Booth multiplication

The ALU receives operands from registers and executes the operation selected by opcode.



Registers

Registers were used to store:

- Input operands
- Temporary data
- Intermediate results
- Final ALU outputs

Registers improve processor speed because they allow fast data access.

Instruction Register (IR)

The Instruction Register stores the currently executing instruction.

The instruction contains:

- Opcode
- Operand fields
- Control information

The opcode is decoded by the control unit.

Control Unit

The control unit generates all necessary control signals.

Responsibilities include:

- Instruction decoding
- Register loading
- ALU selection
- Memory control
- Accumulator control
- Clock synchronization

Example Instruction Types

Instruction	Function
ADD	Addition
SUB	Subtraction
XOR	Bitwise XOR
INC	Increment
DEC	Decrement
SHL	Shift Left
SHR	Shift Right
MUL	Booth Multiplication

The instruction set was designed to keep the architecture simple while still demonstrating important processor concepts.

ALU Design and Operations

The ALU was one of the most important and challenging components of the project.

Addition

The addition circuit performs binary addition using combinational logic.

Subtraction

Subtraction was implemented using two's complement representation.

XOR Operation

The XOR operation performs bitwise exclusive OR between operands.

Increment and Decrement

Increment and decrement operations modify values by +1 and -1 respectively.

Shift Operations

Shift operations were integrated to support bit manipulation.

The implementation of shifting introduced several design challenges because shift circuits require synchronization with register loading and clock behavior.

Booth Multiplication

The Booth Multiplier was integrated later into the processor.

The multiplication circuit:

- Uses signed multiplication
- Supports arithmetic shifting
- Reduces repeated addition operations
- Produces larger result outputs

Integrating the multiplication circuit into the existing architecture required significant modification of control signals and accumulator loading.

ROM-Based Control Unit

One of the most important parts of the project was the ROM-based control unit.

Instead of manually wiring every instruction execution path separately, ROM was used to store micro-operations.

Advantages of ROM-Based Control

- Easier instruction control
- Simplified debugging
- Organized execution flow
- Flexible instruction expansion
- Better visualization of micro-operations

Each ROM location generated control signals for a specific execution cycle.

The ROM outputs controlled:

- Register loading
- ALU operation selection
- Accumulator enable
- Data transfer
- Multiplication execution

ROM Bits	Decoder Output	Control Function	Microoperation
00000	0	—	None
00001	1	d_1	$MAR \leftarrow PC$
00010	2	c_1	$PC \leftarrow PC + 1$
00011	3	R	$B \leftarrow M$
00100	4	c_2	$PC \leftarrow B(AD)$
00101	5	d_2	$MAR \leftarrow B(AD)$
00110	6	r	$A \leftarrow A \wedge B$
00111	7	p	$A \leftarrow A + B, E \leftarrow \text{carry}$
01000	8	a_1	$A \leftarrow A \vee B$
01001	9	a_2	$A \leftarrow A - B$
01010	10	b_1	$B \leftarrow A$
01011	11	W	$M \leftarrow B$
01100	12	b_2	$B \leftarrow B + 1$
01101	13	h	$B(AD) \leftarrow PC, B(OP) \leftarrow 0101, PC \leftarrow MAR$
01110	14	LdAfrB	$A \leftarrow B$
01111	15	HLT	Halt Processor
10000	16	XOR	$A \leftarrow A \oplus B$
10001	17	INC	$A \leftarrow A + 1$
10010	18	DEC	$A \leftarrow A - 1$
10011	19	SHL	$A \leftarrow \text{SHL } A$
10100	20	SHR	$A \leftarrow \text{SHR } A$
10101	21	MUL	$A \leftarrow A \times B$
10110	22	RST	Reset

Booth Multiplication Integration

Integrating Booth Multiplication into the processor was one of the most difficult stages of the project.

Initially, the multiplier circuit worked independently. However, integrating it with the processor created several synchronization and control issues.

Challenges During Integration

- Result not loading into accumulator immediately
- Control signal timing mismatch
- Delay between multiplication completion and accumulator loading
- Clock synchronization issues
- Data path conflicts

Solutions Applied

- Additional clock synchronization logic was introduced.
- Control sequencing was modified.
- ROM micro-operations were updated.
- Accumulator loading timing was adjusted.
- Proper enable signals were added.

After debugging and redesign, the multiplication result was successfully transferred into the accumulator.

Clock and Timing Mechanism

The processor uses clock pulses for sequential operation.

Every instruction execution depends on proper timing.

Role of Clock

- Synchronizes operations
- Controls register loading

- Coordinates ALU execution
- Controls ROM sequencing

Timing Problems Encountered

One of the major problems during development was that some operations executed correctly internally but failed to load results into the accumulator during the first clock cycle.

This issue occurred mainly during multiplication integration.

The problem was solved through:

- Better clock control
- Delay adjustments
- Correct load timing
- Proper sequencing of control signals

This stage provided practical understanding of hardware timing behavior.

Problems Faced During Development

During the implementation process, multiple technical problems were encountered.

Accumulator Loading Problem

The multiplication result was generated correctly but was not loaded into the accumulator immediately.

Cause

Timing mismatch between result generation and accumulator load signal.

Solution

Clock synchronization and control logic modification.

ROM Addressing Complexity

As more instructions were added, ROM organization became more complex.

Cause

Increasing number of micro-operations.

Solution

Careful ROM mapping and opcode organization.

Instruction	ROM Address	0	1	2	3	4	5	6	7	Microoperations	Next Address
FETCH	00000000	0	0	0	0	1	0	0	1	MAR $\leftarrow$ PC	CAR $\leftarrow$ CAR + 1
	00000001	0	0	0	1	1	0	0	1	B $\leftarrow$ M	CAR $\leftarrow$ CAR + 1
	00000010	0	0	0	1	0	1	0	1	PC $\leftarrow$ PC + 1	CAR $\leftarrow$ 2 ^b (OP)+3
	00000011	0	0	0	0	0	0	1	0	NOP	CAR $\leftarrow$ 0
AND	00000100	0	0	1	0	1	0	0	1	MAR $\leftarrow$ B(AD)	CAR $\leftarrow$ CAR + 1
	00000101	0	0	0	1	1	0	0	1	B $\leftarrow$ M	CAR $\leftarrow$ CAR + 1
	00000110	0	0	1	1	0	0	0	1	A $\leftarrow$ A $\wedge$ B	CAR $\leftarrow$ CAR + 1
	00000111	0	0	0	0	0	0	1	0	NOP	CAR $\leftarrow$ 0
ADD	00001000	0	0	1	0	1	0	0	1	MAR $\leftarrow$ B(AD)	CAR $\leftarrow$ CAR + 1
	00001001	0	0	0	1	1	0	0	1	B $\leftarrow$ M	CAR $\leftarrow$ CAR + 1
	00001010	0	0	1	1	1	0	0	1	A $\leftarrow$ A + B, E $\leftarrow$ carry	CAR $\leftarrow$ CAR + 1
	00001011	0	0	0	0	0	0	1	0	NOP	CAR $\leftarrow$ 0
STO	00001100	0	0	1	0	1	0	0	1	MAR $\leftarrow$ B(AD)	CAR $\leftarrow$ CAR + 1

	00001101	0 1 0 1 0 0 0 1	$B \leftarrow A$	$CAR \leftarrow CAR + 1$
	00001110	0 1 0 1 1 0 0 1	$M \leftarrow B$	$CAR \leftarrow CAR + 1$
	00001111	0 0 0 0 0 0 1 0	NOP	$CAR \leftarrow 0$
OR	00010000	0 0 1 0 1 0 0 1	$MAR \leftarrow B(AD)$	$CAR \leftarrow CAR + 1$
	00010001	0 0 0 1 1 0 0 1	$B \leftarrow M$	$CAR \leftarrow CAR + 1$
	00010010	0 1 0 0 0 0 0 1	$A \leftarrow A \vee B$	$CAR \leftarrow CAR + 1$
	00010011	0 0 0 0 0 0 1 0	NOP	$CAR \leftarrow 0$
SUB	00010100	0 0 1 0 1 0 0 1	$MAR \leftarrow B(AD)$	$CAR \leftarrow CAR + 1$
	00010101	0 0 0 1 1 0 0 1	$B \leftarrow M$	$CAR \leftarrow CAR + 1$
	00010110	0 1 0 0 1 0 0 1	$A \leftarrow A - B$	$CAR \leftarrow CAR + 1$
	00010111	0 0 0 0 0 0 1 0	NOP	$CAR \leftarrow 0$
BUN	00011000	0 0 1 0 0 0 0 1	$PC \leftarrow B(AD)$	$CAR \leftarrow CAR + 1$
	00011001	0 0 0 0 0 0 1 0	NOP	$CAR \leftarrow 0$
	00011010	0 0 0 0 0 0 1 0	NOP	$CAR \leftarrow 0$
	00011011	0 0 0 0 0 0 1 0	NOP	$CAR \leftarrow 0$
LOAD	00011100	0 0 1 0 1 0 0 1	$MAR \leftarrow B(AD)$	$CAR \leftarrow CAR + 1$
	00011101	0 0 0 1 1 0 0 1	$B \leftarrow M$	$CAR \leftarrow CAR + 1$

	00011110	0 1 1 1 0 0 0 1	$A \leftarrow B$	$CAR \leftarrow CAR + 1$
	00011111	0 0 0 0 0 0 1 0	NOP	$CAR \leftarrow 0$
HLT	00100000	0 1 1 1 1 0 1 0	Halt	$CAR \leftarrow 0$
	00100001	0 0 0 0 0 0 1 0	NOP	$CAR \leftarrow 0$
	00100010	0 0 0 0 0 0 1 0	NOP	$CAR \leftarrow 0$
	00100011	0 0 0 0 0 0 1 0	NOP	$CAR \leftarrow 0$
XOR	00100100	0 0 1 0 1 0 0 1	$MAR \leftarrow B(AD)$	$CAR \leftarrow CAR + 1$
	00100101	0 0 0 1 1 0 0 1	$B \leftarrow M$	$CAR \leftarrow CAR + 1$
	00100110	0 1 0 0 0 1 0 1	$A \leftarrow A \oplus B$	$CAR \leftarrow CAR + 1$
	00100111	0 0 0 0 0 0 1 0	NOP	$CAR \leftarrow 0$
INC	00101000	0 1 1 0 0 0 0 1	$A \leftarrow A + 1$	$CAR \leftarrow CAR + 1$
	00101001	0 0 0 0 0 0 1 0	NOP	$CAR \leftarrow 0$
	00101010	0 0 0 0 0 0 1 0	NOP	$CAR \leftarrow 0$
	00101011	0 0 0 0 0 0 1 0	NOP	$CAR \leftarrow 0$
DEC	00101100	0 1 0 0 1 0 0 1	$A \leftarrow A - 1$	$CAR \leftarrow CAR + 1$
	00101101	0 0 0 0 0 0 1 0	NOP	$CAR \leftarrow 0$
	00101110	0 0 0 0 0 0 1 0	NOP	$CAR \leftarrow 0$
	00101111	0 0 0 0 0 0 1 0	NOP	$CAR \leftarrow 0$
SHL	00110000	0 1 0 1 0 0 0 1	$A \leftarrow SHL A$	$CAR \leftarrow CAR + 1$
	00110001	0 0 0 0 0 0 1 0	NOP	$CAR \leftarrow 0$
	00110010	0 0 0 0 0 0 1 0	NOP	$CAR \leftarrow 0$

	00110011	0 0 0 0 0 0 1 0	NOP	CAR $\leftarrow$ 0
SHR	00110100	0 1 0 1 1 0 0 1	A $\leftarrow$ SHR A	CAR $\leftarrow$ CAR + 1
	00110101	0 0 0 0 0 0 1 0	NOP	CAR $\leftarrow$ 0
	00110110	0 0 0 0 0 0 1 0	NOP	CAR $\leftarrow$ 0
	00110111	0 0 0 0 0 0 1 0	NOP	CAR $\leftarrow$ 0
MUL	00111000	0 0 1 0 1 0 0 1	MAR $\leftarrow$ B(AD)	CAR $\leftarrow$ CAR + 1
	00111001	0 0 0 1 1 0 0 1	B $\leftarrow$ M	CAR $\leftarrow$ CAR + 1
	00111010	0 1 1 0 1 0 0 1	A $\leftarrow$ A $\times$ B	CAR $\leftarrow$ CAR + 1
	00111011	0 0 0 0 0 0 1 0	NOP	CAR $\leftarrow$ 0

Shift Operation Integration

Shift operations created additional complexity because shifting required sequential behavior.

Cause

Interaction between shift register behavior and ALU execution.

Solution

Separate shift control logic and synchronized execution.

Opcode Expansion Problem

Adding new instructions required modification of opcode structure.

Cause

Limited opcode organization.

Solution

Redesigning instruction mapping and ROM entries.

Booth Multiplier Synchronization

The Booth multiplier worked independently but failed during processor integration.

Cause

Mismatch between multiplier output timing and processor control signals.

Solution

Control signal redesign and execution sequencing.

Clock Dependency Issues

Several operations behaved differently depending on clock order.

Cause

Incorrect sequential triggering.

Solution

Proper clock edge management and execution ordering.

Solutions and Design Improvements

Several improvements were implemented to stabilize the processor.

Improvements Included

- Better ROM organization
- Cleaner control logic
- Improved accumulator loading
- Synchronization fixes
- Optimized opcode arrangement
- Improved shift execution

- Successful multiplier integration

These improvements significantly increased processor stability and functionality.

Working Procedure of the Processor

The processor executes instructions through the following steps:

Step 1: Instruction Fetch

The instruction is fetched from memory into the Instruction Register.

Step 2: Instruction Decode

The opcode is decoded by the control unit.

Step 3: Control Signal Generation

ROM generates the required micro-operations.

Step 4: Operand Transfer

Operands are transferred from registers.

Step 5: ALU Execution

The ALU performs the selected operation.

Step 6: Result Storage

The output is stored in the accumulator.

Step 7: Next Instruction Execution

The processor moves to the next instruction cycle.

This process repeats continuously.

Advantages of the System

The developed processor has several advantages:

- Practical implementation of computer architecture concepts
- ROM-based organized control mechanism
- Expandable instruction set

- Support for arithmetic and logical operations
 - Booth multiplication support
 - Modular design approach
 - Educational value for processor understanding
-

Limitations

Although the processor successfully executes multiple operations, several limitations remain.

Current Limitations

- Simplified processor architecture
- Limited instruction set
- Limited memory organization
- No pipelining
- No cache memory
- No interrupt handling
- Limited data width
- Sequential execution only

These limitations exist because the project was designed mainly for educational purposes.

Future Improvements

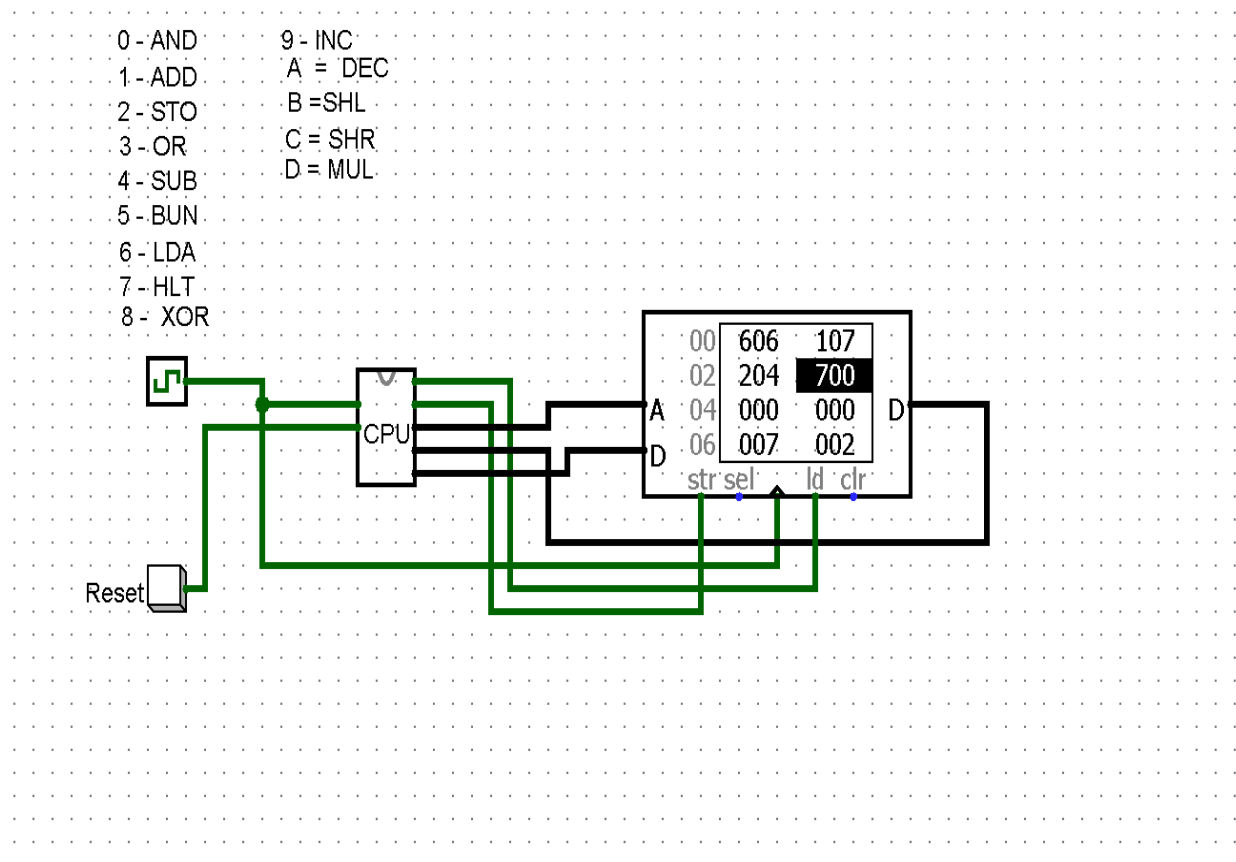
The processor can be further improved in several ways.

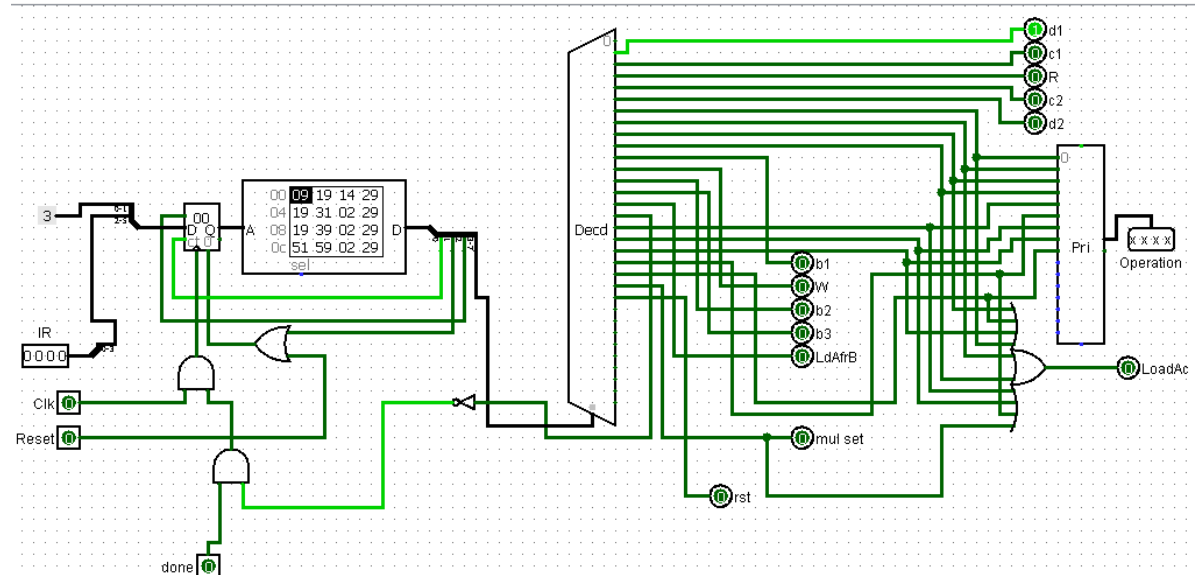
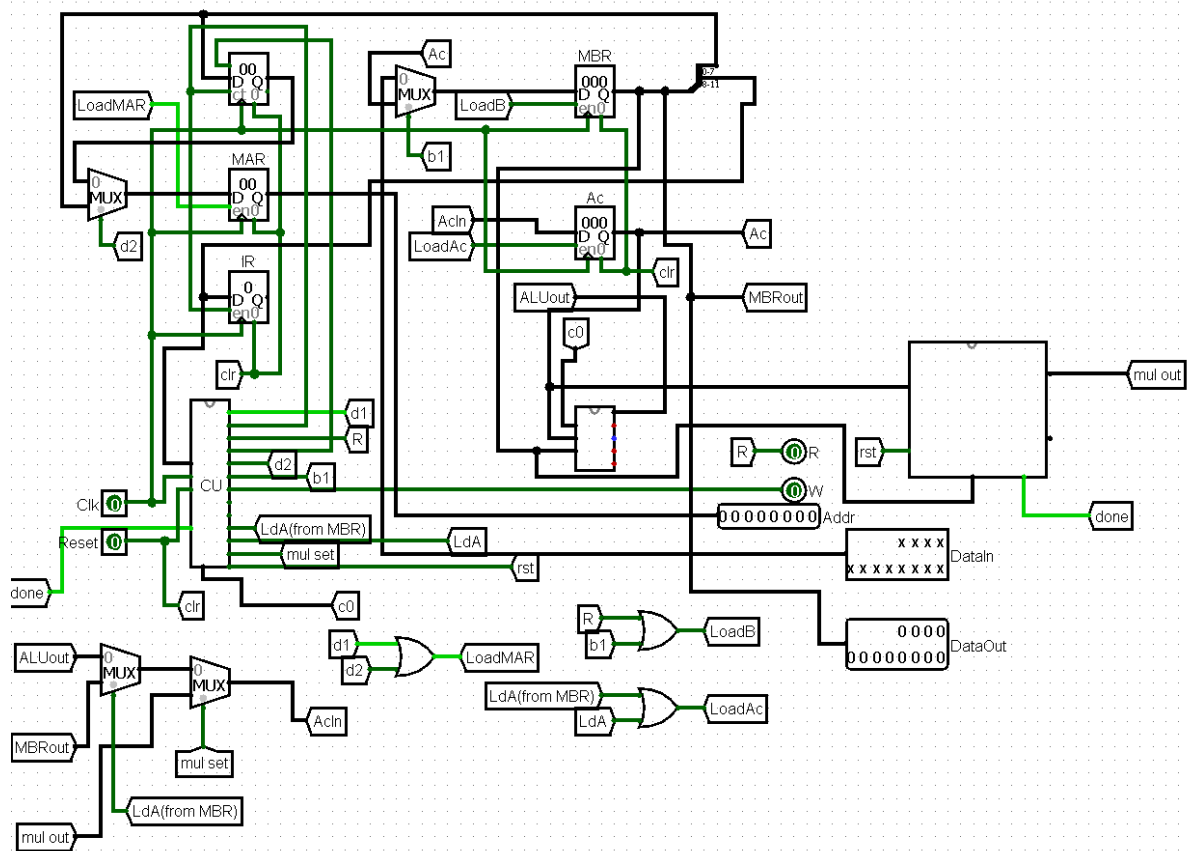
Possible Future Enhancements

- Pipelined architecture
- Cache memory integration
- Floating point operations
- Interrupt system

- Better memory management
- Extended instruction set
- Branching and jump instructions
- Larger register file
- More advanced multiplication algorithms
- FPGA implementation

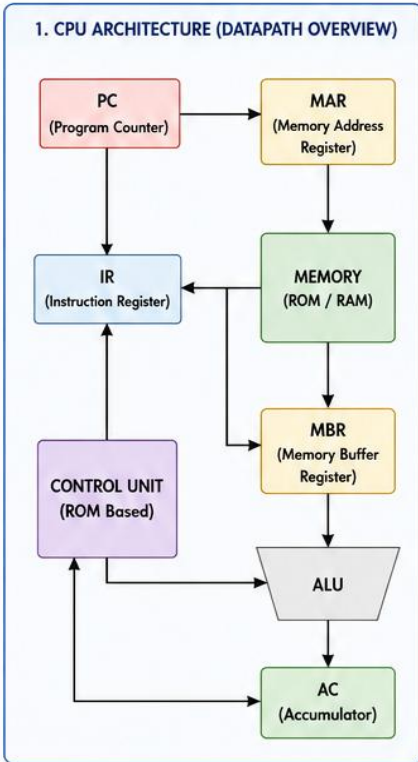
These improvements would make the processor more similar to modern CPU architectures.



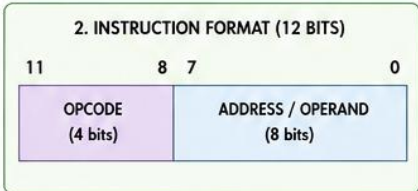


CUSTOM CPU – FETCH, DECODE, EXECUTE FLOWCHART

1. CPU ARCHITECTURE (DATAPATH OVERVIEW)



2. INSTRUCTION FORMAT (12 BITS)



3. INSTRUCTION SET (EXAMPLE)

Opcode (Hex)	Instruction	Operation
0x1	LOAD addr	$AC \leftarrow M[addr]$
0x2	STORE addr	$M[addr] \leftarrow AC$
0x3	ADD addr	$AC \leftarrow AC + M[addr]$
0x4	SUB addr	$AC \leftarrow AC - M[addr]$
0x5	SHIFT addr	$AC \leftarrow \text{Shift } AC$
0x6	XOR addr	$AC \leftarrow AC \text{ XOR } M[addr]$
0xF	HLT	Stop Execution

START

FETCH CYCLE

T0

$MAR \leftarrow PC$

T1

$MBR \leftarrow M[MAR]$
 $PC \leftarrow PC + 1$

T2

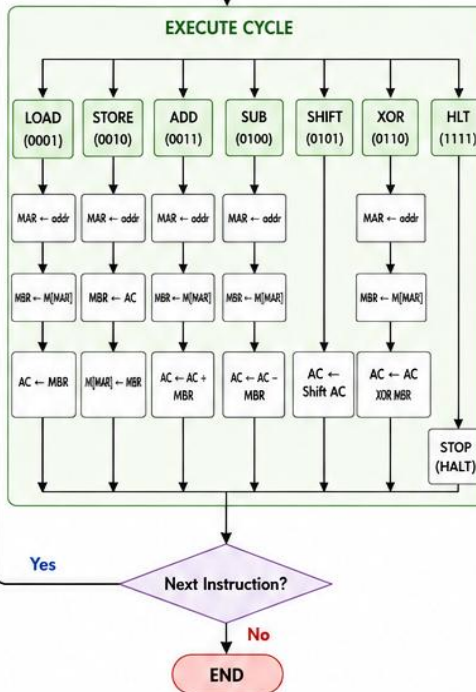
$IR \leftarrow MBR$

DECODE

Decode OPCODE from $IR[11:8]$
Get ADDRESS from $IR[7:0]$

Which Instruction?

EXECUTE CYCLE



Yes

Next Instruction?

No

END

5. FETCH CYCLE (DETAIL)

Step	Micro-Operation	Description
T0	$MAR \leftarrow PC$	Copy PC to MAR
T1	$MBR \leftarrow M[MAR]$ $PC \leftarrow PC + 1$	Read memory and increment PC
T2	$IR \leftarrow MBR$	Load instruction into IR

6. EXECUTION EXAMPLE (ADD addr)

Step	Micro-Operation
T3	$MAR \leftarrow \text{addr}$
T4	$MBR \leftarrow M[MAR]$
T5	$AC \leftarrow AC + MBR$

7. CONTROL UNIT (ROM)

- ROM stores control words for each timing step.
- Control signals are generated based on:
 - Current state (T0, T1, T2, ...)
 - Instruction (opcode)
- Outputs control signals to: Registers, ALU, Memory, Bus, etc.

FLOW SUMMARY

PC points to next instruction → Fetch instruction from memory → Decode opcode and address → Execute operation using ALU, Registers and Memory → Repeat until HLT.

DISCUSSION :

The microprogrammed processor designed in this project is based on a custom 12-bit computer architecture implemented using Logisim Evolution. The processor utilizes a ROM-based control unit where each instruction is executed through a sequence of microoperations stored in control memory. A microprogram control mechanism was developed to generate the required control signals for instruction execution, enabling systematic coordination between the Arithmetic Logic Unit (ALU), registers, memory interface, and data bus system. The architecture includes essential components such as the Program Counter (PC), Memory Address Register (MAR), Instruction Register (IR), Accumulator Register (A), General Purpose Register (B), Carry Flag (E), Control Address Register (CAR), RAM, ROM, and a shared bus structure for data transfer.

The processor supports a complete fetch-decode-execute cycle controlled through microinstructions stored in ROM. During the fetch phase, the processor transfers the Program Counter value to the Memory Address Register, reads the instruction from memory, increments the Program Counter, and decodes the opcode to determine the starting microaddress of the corresponding instruction routine. Each instruction is executed through multiple clock-driven microoperations controlled by the microprogrammed control unit.

Several arithmetic, logical, memory-reference, and branching instructions were implemented in the processor design. Basic instructions include AND, ADD, SUB, OR, LOAD, STORE, unconditional branching (BUN), and HLT operations. In addition to the standard instruction set, the processor was further extended with advanced operations such as XOR, Increment (INC), Decrement (DEC), Shift Left (SHL), Shift Right (SHR), and Booth-based Multiplication (MUL). These instructions significantly enhanced the computational capability of the processor and required additional control logic, ALU functionality, and ROM microprogram modifications.

The Arithmetic Logic Unit was designed to support multiple arithmetic and logical operations through a control-select mechanism. Separate control signals were generated for addition, subtraction, logical AND, logical OR, XOR, increment, decrement, and shift operations. Shift operations were integrated using dedicated shift circuitry connected with the accumulator register. The multiplication operation was implemented using Booth's multiplication algorithm, allowing signed binary multiplication within the processor architecture.

The control unit of the processor was implemented using a microprogrammed ROM architecture instead of hardwired control logic. Each ROM address contains a microinstruction composed of multiple control bits responsible for activating specific microoperations within the processor datapath. An 8-bit control field was used to generate control signals, while the ROM addresses were organized sequentially for instruction execution. Each instruction occupied four ROM locations, including unused states filled with NOP microoperations for synchronization and timing consistency.

During the development process, several practical implementation challenges were encountered. These included synchronization problems between registers and clock pulses, accumulator loading delays, control signal conflicts, ROM addressing complexity, integration of multiplication hardware, and shift operation timing issues. These problems were systematically analyzed and resolved through debugging, redesign of timing sequences, modification of ROM microinstructions, and optimization of control signal generation. Special attention was given to ensuring proper clock synchronization between the ALU, registers, and memory system.

The processor was fully simulated and verified using Logisim Evolution. Multiple test programs were executed to validate arithmetic operations, logical computations, memory access instructions, branching operations, shifting functions, and multiplication functionality. Simulation results confirmed that the processor successfully executed instructions according to the designed microprogram sequences and produced correct outputs for all implemented operations.

Overall, this project demonstrates the practical design and implementation of a microprogrammed processor architecture using digital logic design principles. The project provides a comprehensive understanding of processor organization, datapath design, control unit implementation, microprogramming techniques, ROM-based control logic, ALU integration, memory interfacing, and instruction execution mechanisms. The completed processor serves as a functional educational model for studying computer organization and architecture concepts in detail.

Conclusion

This project successfully demonstrated the design and implementation of a custom processor architecture using Logisim Evolution.

The project provided practical experience with:

- Processor organization
- ALU design
- Instruction execution
- ROM-based control logic
- Micro-operations
- Clock synchronization
- Register transfer operations
- Booth multiplication integration

Throughout the project, several real hardware-level problems were encountered and solved. These debugging experiences significantly improved understanding of processor timing, control sequencing, and hardware behavior.

The successful integration of arithmetic, logical, shift, and multiplication operations demonstrated that the processor architecture was functionally complete and capable of executing multiple instruction types.

This project not only strengthened theoretical understanding of Computer Architecture but also provided valuable practical implementation experience.

References

- [1] William Stallings, *Computer Organization and Architecture: Designing for Performance*, 8th Edition, Pearson Education, 2010. [filecite2turn0file1](#)
- [2] Morris Mano and Michael D. Ciletti, *Digital Design*, 5th Edition, Pearson.
- [3] David A. Patterson and John L. Hennessy, *Computer Organization and Design*, Morgan Kaufmann.
- [4] Douglas V. Hall, *Microprocessors and Interfacing: Programming and Hardware*, McGraw-Hill.
- [5] Booth, A. D., "A Signed Binary Multiplication Technique," *Quarterly Journal of Mechanics and Applied Mathematics*, Oxford University Press.
- [6] Logisim Evolution Documentation and Simulation Environment.
- [7] Course materials and lecture notes on Computer Architecture. [filecite2turn0file0](#)